

Def :

$$E(2) \cong (R^2, +) \rtimes O(2)$$

$(R^2, +)$:

① 2 d vector , e.g. $\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$

② operation $+$, i.e. $\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} x_1 + y_1 \\ x_2 + y_2 \end{pmatrix}$

③ $\star$ acting on the point in the space : $\vec{a} = \begin{pmatrix} a_1 \\ a_2 \end{pmatrix}$

$$\begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \xrightarrow{\vec{a}} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \vec{a} = \begin{pmatrix} x_1 + a_1 \\ x_2 + a_2 \end{pmatrix}$$

$O(2)$

① 2 d matrix $\begin{pmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \end{pmatrix}$ with condition : $O^T O = I_2$

② operation : matrix multiplication

③ acting on the point in the space :

matrix multiplication

X :

Say $(t_1, O_1), (t_2, O_2) \in E(2)$, $t \in (\mathbb{R}^2, +)$, $O \in O(2)$

(we also denote (t_1, O_1) as $t_1 O_1$ for simplicity, but note

then define the group operation of $E(2)$: $t_1 O_1 \cdot t_2 O_2$
 $\neq t_1 \cdot t_2 O_1$

$$(t_2, O_2) \cdot (t_1, O_1) = (O_2 t_1 + t_2, O_2 O_1)$$

We can prove

$E(2)$ is the group containing all transformation

that keep distance invariant.

Note:

$C_N, D_N, SO(2)$ is the subgroup of $O(2)$.

C_N : rotation by given angle once a time

D_N : C_N & reflection

Now we have group $E(2)$, denote element of $E(2)$ as g .

Then we illustrate how group acts on the data set.

First we introduce the figure of data set

① scalar case

1	0	2	3	4
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.

group transformation

$$\varphi'(x) = \varphi(g^{-1}x)$$

$$\Leftrightarrow T(g)[\varphi(x)] = \varphi(g^{-1}x)$$

$\varphi(x) \rightarrow x$ in this case is the index, related to the

② vector case

coordinate in the R^2 space. x is grid.

Vector	.	.	.	.
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.

i.e., $x = \begin{pmatrix} 1, 1 \\ 1, 2 \end{pmatrix}$

first line
first column

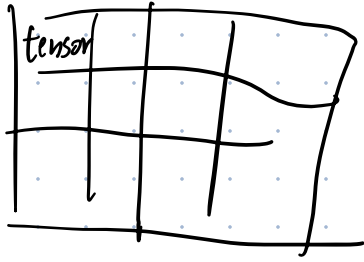
$$V'(x) = \rho(g) V(g^{-1}x)$$

$$\Leftrightarrow T(g)[V(x)] = \rho(g) V(g^{-1}x)$$

$\rho(g)$: group representation (can be proved)

② Tensor case

문제는 4x4?



→ T - - - similar - -

(→ In principle tensors don't have essential difficulties for clarity we skip this complicated case.
by rewrite tensor as direct sum)

General input : the direct sum of the

above one :

$$\varphi_1(x) \oplus \dots \oplus V_1(x) \oplus \dots \oplus T_1(x) \oplus \dots$$

They are independent when do convolution.

(e.g. :

$$\begin{array}{ccc} \text{input:} & \text{convolution} & \text{output} \\ \varphi_{in}(x) \oplus V_{in}(x) & \longrightarrow & \varphi_{out}(x) \oplus V_{out}(x) \end{array}$$

The restriction of convolution layer :

(in continuous case)

Define:

$$f_{out}(x) = (k * f_{in})(x) = \int k(y) f_{in}(x+y) dy$$

→ for vector: $k = \begin{bmatrix} k(x) & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$ $f = \text{vector}$

$$k(x) = R^{in \times out}$$

→ for scalar: $k = \begin{bmatrix} k(x) & - & - \\ - & - & - \\ - & - & - \end{bmatrix}$ $f = \text{field}$

$k(x) = R$

x : index of matrix G

After some calculation, we find the restriction to $k(x)$.

$$k(gx) = P_{out}(g) k(x) P_{in}(g^{-1}) \quad \textcircled{1}$$

recall P is the rep of group (Matrix)

From group theory, clearly we can write

ρ as :

$$\rho = Q^{-1} \left[\bigoplus_i \psi_i \right] Q$$

ψ_i : irreps Q : similarity trans. matrix .

①

$$\Rightarrow Q_{out} k(gx) Q_{in}^{-1} = \left[\bigoplus \psi_i \right] Q_{out} k Q_{in}^{-1} \left[\bigoplus \psi_j \right]$$

define $Q_{out} k Q_{in}^{-1} = K$

$$\Rightarrow K(gx) = \left[\bigoplus \psi_i \right] k \left[\bigoplus \psi_j \right]$$

$$= \begin{bmatrix} \psi_1 & & \\ & \psi_2 & \\ & & \ddots \\ & & & \psi_n \end{bmatrix} k \begin{bmatrix} \psi'_1 & & \\ & \psi'_2 & \\ & & \ddots \\ & & & \psi'_n \end{bmatrix}$$

writing in index version

/

$$k^{ij}(gx) = \psi_i k^{ij}(x) \psi_j^{-1}(g) \quad k^{ij} := \text{Cent } kQ_{im}^{-1}$$

Thus we decompose equ. ① into some easy to solving ones.

Then there's an example computing basis of $O(2)$ (subgroup of $E(2)$)

Skip it - -

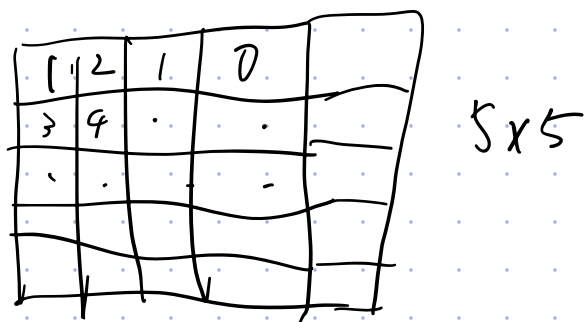
① sufficient?

② regular rep., quotient rep., induced rep. to be ...

pooling, max pooling, ReLU $\rightarrow$ invariant under ...

Review standard CNN:

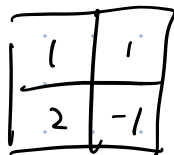
Input: grey picture $\leftrightarrow$ scalar field



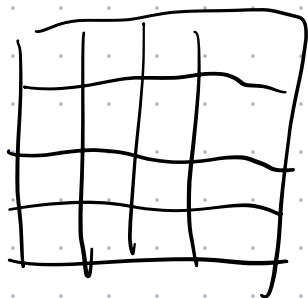
convolution



kernel



2x2



out put

4x4 scalar field



4x4

pooling (2x2) (optional)

2x2

x N



↓ train

classification mode

can be configured
each layer

Equivariant network ENN, Assume C_4 , reg rep, 1 layer.

Input: 5x5 scalar:

convolution
matrix
multiplication

* kernel: not a scalar field
but (1x4 vector field)

↳ depend on choice
of group and
rep.



output : 4×4 vector ^(1x4) field

$\begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$	$\begin{pmatrix} 1 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 1 \end{pmatrix}$
$\begin{pmatrix} 1 \\ 1 \end{pmatrix}$	$\begin{pmatrix} 1 \\ 1 \end{pmatrix}$	.	.
.	.	.	.
.	.	.	.

↓ ReLU (element wise)

4×4 vector ^(1x4) field

example :

$\begin{pmatrix} 1 \\ -1 \end{pmatrix}$	$\begin{pmatrix} -1 \\ -2 \end{pmatrix}$
$\begin{pmatrix} 2 \\ -1 \end{pmatrix}$	$\begin{pmatrix} 2 \\ 2 \end{pmatrix}$

→

$\begin{pmatrix} 1 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 0 \\ 0 \end{pmatrix}$
$\begin{pmatrix} 2 \\ 0 \end{pmatrix}$	$\begin{pmatrix} 2 \\ 2 \end{pmatrix}$

↓ pooling (optional), same idea!

example :

2×2 vector field

$\begin{pmatrix} 100 \\ 4 \end{pmatrix}$	$\begin{pmatrix} 111 \\ 3 \end{pmatrix}$
$\begin{pmatrix} 121 \\ 2 \end{pmatrix}$	$\begin{pmatrix} 131 \\ 1 \end{pmatrix}$

2×2

→

$\begin{pmatrix} 131 \\ 4 \end{pmatrix}$
--

1×1

xN layer
(one can choose
different group
and rep
or not)

group pooling acting on vector:

example:

2x2
Scalar field

$\begin{pmatrix} 1 \\ 2 \end{pmatrix}$	$\begin{pmatrix} 2 \\ 3 \end{pmatrix}$
$\begin{pmatrix} 1 \\ 3 \end{pmatrix}$	$\begin{pmatrix} 2 \\ 4 \end{pmatrix}$



2	3
3	4

intuitive
extract the

flatten

1x4



classifier

Another difference:

CNN: every elements in k matrix is learnable
parameter

ENN: restricted to the required symmetry group and rep.

k is decomposed into a series of basis,

only the coefficient in front of the basis are learnable.

are calculated beforehand and

Basis encoded in the program, so we don't need to deal with it directly.

In practical:

① set up group (in our case C_4 or C_8)

and rep. (in our case reg.)

then the program assign basis automatically

Since it coheres with ReLU)

less ability
precise, ↑
approx., more powerful
↑ ↑

② set up input, ReLU, number of layer, pooling.
like standard CNN

③ set up group pooling

④ use output (and inverse propagation) train classifier, and the kernel,
(like standard CNN)

Expect:

ENN more expressive than CNN,

(i.e. less training time to be more precise in distinguishing
rotated handwriting numbers. --)